Workshop 7: Synchronization

Remember: Discuss each question with your parnter and write down a short answer. Writing is important to force you to really think about whether your analysis was right when you do the experiment later. *Downloads:* See the Workshop info on the class website.

Part 0 - What was confusing

Preparation

- Each person writes down what they wrote in the online reading summary for what was confusing or a question they had about the reading material, or something else they find confusing now.
- Write your selection here:

Group

- Go around the table and each person presents their point.
- Discuss which one the table finds most interesting.
- Write the table's selection here:
- Discuss how to explain it so it is not confusing, or answer it if it is a question, or what more information/help you would need to do so if you cannot.
- Write the table's answer here:

Presenting

- Choose someone from the group to present it to the class.
- Pesenter:
- Now wait for the TA to call on your group and have your presenter present.

For the rest of the questions you work at your own pace with your partner. Try to get as much of the workshop done as you can, but there is no requirement to finish it all.

If you need help or have questions, feel free to ask other groups or the TA. Don't forget to sign in for attendance before you leave.

Part 1 - Concurrency tradeoffs

Preparation

- Download the code for `threads-bugs/` from the class website (already provided in this workshop's folder).
- Open the book **Chapter 32** and go to the Homework (Code) section.
- Build the code: `cd threads-bugs && make`.
- **Note:** `vector-nolock.c` is the book's original code and uses x86-specific inline assembly (`lock; xaddl`) for its fetch-and-add. It will fail to compile on Apple Silicon (arm64) Macs with an "invalid output constraint" error - this is expected, not something to debug, and `make` is set up so every other program still builds first. Use `vector-nolock-portable.c` instead (also provided, and built by `make`) for Questions 9-10 - it is identical except its `fetch_and_add()` uses

the `__sync_fetch_and_add()` GCC/Clang builtin instead of inline assembly, so it compiles and behaves the same on both x86 (Linux/Intel Mac) and arm64 (Apple Silicon Mac).

Question 0: What do the flags do, and what makes `vector_add()` deadlock-prone?

- This code is organized somewhat strangely. Each file (e.g., `vector-deadlock.c`) contains just a piece of the program. Look at `vector-deadlock.c`. What part of the program does it contain?
-
- What does the `#include "main-common.c"` do? (Hint: take a look at the file `main-common.c` and think about what you would get if you put `vector-deadlock.c` right before it in one file…)
-
- From this you can see that the different program files are just different definintions for `vector_add()` and `fini()`.
- Now, what does `-d` do? (you can run the program with -help or just look at the main-common.c code to answer.)
-
- What does `-p` do?
-
- What does `-n` control, and what's the minimum needed to see a deadlock?
-
- What does `-t` show, and what does "Retries" mean in `vector-try-wait`'s output?
-

Questions 1-3 from the book: getting deadlock

- Take a look at `vector-deadlock.c` and figure out (based on the reading) how it can deadlock. Note that the code is locking two vectors before updating the destination vector with their sum. (Hint: how many threads do you need, and what do those threads have to do in an unfortunate order for them to get stuck?)
-
- Try running `./vector-deadlock -d -v -n 2 -l 1` a few times to see if you get a deadlock. Did you? (Hint: how will you know if you have a deadlock? Would typing `control-c` in that situation help?)
-
- Now try to figure out how to get a deadlock reliably by increasing the number of times the program tries to add the vectors (e.g., multiple the number after -l by 10x until you get deadlocks reliably.). How many iterations did you need?
-
- Now increase the number of threads by 10x (-n 20) and reduce the number of iterations to see how many iterations you need for deadlock.
-
- Explain why increasing the number of iterations or the number of threads increases the likelihood of deadlock.
-

Question 4 from the book: Global order

- Look at `vector-global-order.c`. How does this make sure the locks are always taken in a global order?
-
- Why is the second unlock in the code conditional? E.g., why does it not always unlock both the source and destination vectors?
-
- Verify that this version does not deadlock for the parameters you found did deadlock in the `vector-deadlock` version.

Question 7 from the book: Try and Wait

- The `vector-try-wait.c` implementation tries to get the destination vector lock, and then, if it succeeds, it tries to get the source vector lock. But, if it can't get the source vector lock, it then immediately unlocks the destination vector lock one and tries the whole thing again. How does this avoid deadlock?
-
- Verify that this version does not deadlock for the parameters you found did deadlock in the `vector-deadlock` version.
- How is the perforamnce of this version? How many retries do you get?
-

Question 8 from the book: Avoiding wait an hold with a global lock

- The `vector-avoid-hold-and-wait.c` implementation uses a single global lock to make sure that only one thread can "start" at a given time. This ensures that two threads can't grab locks that the other wants.
- Verify that this version does not deadlock for the parameters you found did deadlock in the `vector-deadlock` version.

Question 9 from the book: Avoiding locks with hardware support

- The `vector-nolock-portable.c` uses the hardware operation "fetch_and_add" which does an atomic load/add/store. This means that the updates cannot be interrupted by other threads.
- Verify that this version does not deadlock for the parameters you found did deadlock in the `vector-deadlock` version.
- What is different about the critical section in this version from the other versions? (Hint: think about what is updated atomically here vs. in the global-order case?) Is this a problem?
-

Question 10 from the book: performance

- Now run each version with 8 threads an 20000 iterations without the `-v` flag and compare the performance. E.g., `./vector-avoid-hold-and-wait -d -n 8 -l 20000 -t` and `./vector-global-order -d -n 8 -l 20000 -t` and `./vector-try-wait -d -n 8 -l 20000 -t` and `./vector-nolock-portable -d -n 8 -l 20000 -t`. What can you say about which is fastest?

Algorithm	Runtime
global order	
try and wait	
avoid hold and wait	
nolock portable	

•

Last question: What about low-contention?

- Re-run each of the above with the `-p` flag. This ensures that each thread is operating only on its own vector. E.g., there is no contention, so there will never be a need for locks! (This is the best-case scenario.) Compare the performance here, and include the `vector-deadlock` version, since it won't deadlock if there can't be deadlock! (If they are too fast, then increase the number of iterations to get a measurement.)

Algorithm	Runtime
deadlock	
global order	
try and wait	
avoid hold and wait	
nolock portable	

- What did you notice about the *avoid hold and wait* one? Why is this the case?
•

Synthesis: comparing all five versions, and connecting back to the chapter

Fill in this table from what you measured (not from the book text):

	Can deadlock?	Scales with no contention (`-p`)?	Main cost
`vector-deadlock`			Deadlocks under real contention
`vector-global-order`			None significant
`vector-try-wait`			Retries explode under real contention
`vector-avoid-hold-and-wait`			Global lock serializes *everyone*, contended or not
`vector-nolock-portable`			Weaker semantics (not atomic across the whole vector)

- The chapter (Section 32.2) also describes two *non-deadlock* bug types: **atomicity violations** (Figure 32.2, e.g. a NULL-checked-then-used pointer that another thread clears in between) and **order violations** (Figure 32.4, e.g. code that assumes a thread has already been initialized). None of the `vector_add()` programs are examples of either of those - they're all about the *third* category, deadlock. Discuss: which category would a bug look like if two threads called `vector_add()` on the *same* vectors with no locking at all (i.e., delete all the locking code)? (A: an atomicity violation - the read-modify-write of each element is no longer atomic, so updates can be lost, the same underlying problem as the un-synchronized counter you saw in Workshop 6, just on a vector instead of a single integer.)

Part 2 - Condition variables (`threads-cv/`)

Preparation

- Download the code for `threads-cv` from the class website (already provided in this workshop's folder).
- Open the book **Chapter 30** and go to the Homework (Code) section.
- Build the code: `cd threads-cv && make`. You should get four executables: `main-one-cv-while`, `main-two-cvs-if`, `main-two-cvs-while`, `main-two-cvs-while-extra-unlock`. (We'll skip the last one for now.)

Question 0: What do the flags and the four programs do?

- Use the README to answer the following questions:

1. What does `-m` control?

2. What do `-p` and `-c` control?

3. What do `-P` and `-C` do, and how is the sleep string written?

4. All four programs implement the same producer/consumer idea. Which one is actually correct?

5. What do `-v` and `-t` show you?

Question 1: What design parameters actually affect performance?

`main-two-cvs-while.c` is the correct version (the other two are both broken in different ways). - We can simulate a consumer doing some real work with a sleep string at a labeled point: `-C 0,0,0,1,0,0,0` sleeps 1 second at `c3` (right after waking up, but *before* releasing the lock); `-C 0,0,0,0,0,0,1` sleeps 1 second at `c6` (right after releasing the lock). Look at the source to confirm this for yourself. - Which do you expect to have worse performance: a consumer that spends time 'working' (simulated with the sleep string) while it holds the lock (`c3`) or after it has released the lock (`c6`) and why? - - Run all four combinations below (one producer, three consumers, 10 items, but with two different buffer sizes and different delays) and record the total time (`-t`):

Sleep point	Buffer size	Command	Runtime
`c3` (still holding the lock)	`-m 1`	`./main-two-cvs-while -p 1 -c 3 -m 1 -C 0,0,0,1,0,0,0:0,0,0,1,0,0,0:0,0,0,1,0,0,0 -l 10 -t`	
`c3` (still holding the lock)	`-m 3`	same command with `-m 3`	
`c6` (lock already released)	`-m 1`	`./main-two-cvs-while -p 1 -c 3 -m 1 -C 0,0,0,0,0,0,1:0,0,0,0,0,0,1:0,0,0,0,0,0,1 -l 10 -t`	
`c6` (lock already released)	`-m 3`	same command with `-m 3`	

- Does this support your analysis above?
-
- How does the buffer size affect the performance based on where the delay is? E.g., does a bigger buffer matter more or less for different delays, and, if so, why?
-
- Note: We'll `-m 1` in the next two questions on purpose: a small buffer makes it flip between empty and full on almost every single item, which means producers and consumers hit the condition variable far more often - exactly the situation that exposes the bugs below.

Question 2: Does the single-CV version actually break?

- Look at `main-one-cv-while.c`. It uses one condition variable for both producers and consumers, instead of the two (`empty`/`fill`) that the correct version uses. Before running anything: with one producer, one consumer, and `-m 1`, could this go wrong? (Hint: if a signal wakes some other thread, what threads could be woken if there is only one producer and one consumer?)
-
- Now add a second consumer and run it several times *without* `-v` (so a hang is obvious): `./main-one-cv-while -p 1 -c 2 -m 1 -l 20`.
-
- How do you know it's actually stuck, rather than just doing a lot of work? (Hint: what would `top`/Activity Monitor show for a hung process vs. a slow one?)
-
- Why it hangs: with two consumers, the producer can fill the buffer, signal (waking one consumer), and loop back around fast enough to find the buffer full again *before* either consumer has actually run - so it goes to sleep too. Now the producer and both consumers can end up asleep on the *same* condition variable. When the one consumer that eventually runs finishes and signals, that signal can't tell "producer" from "other consumer" apart - it may wake the wrong one, which finds nothing to do and goes right back to sleep, leaving everyone asleep forever. This is Figure 30.11 in the book.

Question 3: Does the "if instead of while" version actually break?

- Now look at `main-two-cvs-if.c`, which checks the buffer state with `if` instead of `while` before waiting. Using the same reasoning as Question 2, would you expect `-p 1 -c 1 -m 1` (1 producer and 1 consumer) to break?
-
- Now try `-p 1 -c 2 -m 1 -l 50` (2 consumers) ten times, and separately try `-p 2 -c 1 -m 1 -l 50` (2 producers) ten times. What happens?
-
- How does this failure look different from Question 2's? (Hint: what do you actually see on screen, and how quickly?)
-
- If a program "stopped responding," which of these two bugs would you suspect, and why?
-
- Why it crashes: with `if`, a woken thread does *not* re-check the condition before proceeding - it just assumes the state it waited for is still true. If a second thread of the same role "sneaks in" between the signal and the woken thread actually running, it can grab (or fill) the buffer first. When the woken thread finally runs, it acts on a buffer state that's no longer what it expects, and the `ensure()` check in `do_get()`/`do_fill()` fires. (This is most visible when the buffer size is 1 since a single thread getting in the way is enough to trigger the problem.)

Synthesis: comparing all three versions

Fill in this table from what you just observed (not from the book text):

	Broken with 1p/1c?	Broken with more threads?	How it fails	Root cause
`main-one-cv-while`				Single CV can wake the wrong *role*
`main-two-cvs-if`				`if` instead of `while`: no re-check after waking
`main-two-cvs-while`				Correct: two CVs, `while`, lock held for the whole critical section

- Discuss: both of these bugs only need *more than one thread of the same role* to appear - why does that matter for testing your own concurrent code? (A: testing with just one producer and one consumer, which is often the easiest configuration to write and debug first, can pass every time and still hide a real bug.)
- Discuss: which failure mode would you rather have while debugging - a silent hang or a loud crash - and why?

You're done for the day! *Don't forget to sign the attendance sheet.*